



李宏毅深度学习教程

LeeDL Tutorial

<https://github.com/datawhalechina/leedl-tutorial>

王琦 杨毅远 江季 编著

版本：1.2.4

2025 年 6 月 13 日

前言

李宏毅老师是台湾大学的教授，其《机器学习》（2021 年春）是深度学习领域经典的中文视频之一。李老师幽默风趣的授课风格深受大家喜爱，让晦涩难懂的深度学习理论变得轻松易懂，他会通过很多动漫相关的有趣例子来讲解深度学习理论。李老师的课程内容很全面，覆盖了到深度学习必须掌握的常见理论，能让学生对于深度学习的绝大多数领域都有一定了解，从而可以进一步选择想要深入的方向进行学习，对于想入门深度学习又想看中文讲解的同学是非常推荐的。

本教程主要内容源于《机器学习》（2021 年春），并在其基础上进行了一定的原创。比如，为了尽可能地降低阅读门槛，笔者对这门公开课的精华内容进行选取并优化，对所涉及的公式都给出详细的推导过程，对较难理解的知识点进行了重点讲解和强化，以方便读者较为轻松地入门。此外，为了丰富内容，笔者在教程中选取了《机器学习》（2017 年春）的部分内容，并补充了不少除这门公开课之外的深度学习相关知识。



(a) 京东购买



(b) 当当购买

纸质书购买

致谢

特别感谢 Sm1les、LSGOMYP、FuWeiru对本项目的帮助与支持。

扫描下方二维码，然后回复关键词“李宏毅深度学习”，即可加入“LeeDL-Tutorial 读者交流群”



Datawhale

一个专注于 AI 领域的开源组织

版权声明

本作品采用知识共享署名-非商业性使用-相同方式共享 4.0 国际许可协议进行许可。

作者简介

王琦 上海交通大学人工智能教育部重点实验室博士研究生，硕士毕业于中国科学院大学。Datawhale 成员，《深度学习详解》、《Easy RL: 强化学习教程》作者，英特尔边缘计算创新大使，Hugging Face 社区志愿者，AI TIME 成员。主要研究方向为强化学习、计算机视觉、深度学习。曾获中国光谷·华为杯第十九届中国研究生数学建模竞赛二等奖、中国大学生计算机设计大赛二等奖、亚太地区大学生数学建模竞赛（APMCM）二等奖，发表 NeurIPS、ICLR 等多篇论文，个人主页: <https://qiawang067.github.io/>。

杨毅远 牛津大学计算机系博士研究生，硕士毕业于清华大学。Datawhale 成员，《Easy RL: 强化学习教程》作者。主要研究方向为时间序列、数据挖掘、智能传感系统，深度学习。曾获国家奖学金、北京市优秀毕业生、清华大学优秀学位论文、全国大学生智能汽车竞赛总冠军等荣誉，发表 SCI/EI 论文多篇，个人主页: <https://yyysjz1997.github.io/>。

江季 网易高级算法工程师，硕士毕业于北京大学。Datawhale 成员，《Easy RL: 强化学习教程》作者。主要研究方向为强化学习、深度学习、大模型、机器人等。曾获得国家奖学金、上海市优秀毕业生等荣誉，发表强化学习与游戏 AI 等相关专利多项，个人主页: <https://github.com/JohnJim0816>。

主要符号表

a	标量
$\mathbf{a}$	向量
$\mathbf{A}$	矩阵
$\mathbf{I}$	单位矩阵
$\mathbb{R}$	实数集
$\mathbf{A}^T$	矩阵 $\mathbf{A}$ 的转置
$\mathbf{A} \odot \mathbf{B}$	$\mathbf{A}$ 和 $\mathbf{B}$ 的按元素乘积
$\frac{dy}{dx}$	y 关于 x 的导数
$\frac{\partial y}{\partial x}$	y 关于 x 的偏导数
$\nabla_{\mathbf{x}} y$	y 关于 $\mathbf{x}$ 的梯度
$a \sim p$	具有分布 p 的随机变量 a
$\mathbb{E}[f(x)]$	$f(x)$ 的期望
$\text{Var}(f(x))$	$f(x)$ 的方差
$\exp(x)$	x 的指数函数
$\log x$	x 的对数函数
$\sigma(x)$	Sigmoid 函数, $\frac{1}{1 + \exp(-x)}$
s	状态
a	动作
r	奖励
π	策略
γ	折扣因子
τ	轨迹
G_t	时刻 t 时的回报
$\arg \min_a f(a)$	$f(a)$ 取最小值时 a 的值

目录

第 1 章 机器学习基础	9
1.1 案例学习	9
1.2 线性模型	14
1.2.1 分段线性曲线	16
1.2.2 模型变形	24
1.2.3 机器学习框架	27
第 2 章 实践方法论	29
2.1 模型偏差	29
2.2 优化问题	29
2.3 过拟合	32
2.4 交叉验证	35
2.5 不匹配	37
第 3 章 深度学习基础	39
3.1 局部极小值与鞍点	39
3.1.1 临界点及其种类	39
3.1.2 判断临界值种类的方法	40
3.2 批量和动量	44
3.2.1 批量大小对梯度下降法的影响	45
3.2.2 动量法	49
3.3 自适应学习率	51
3.3.1 AdaGrad	53
3.3.2 RMSProp	55
3.3.3 Adam	57
3.4 学习率调度	57
3.5 优化总结	58
3.6 分类	59
3.6.1 分类与回归的关系	59
3.6.2 带有 softmax 的分类	60
3.6.3 分类损失	61
3.7 批量归一化	62
3.7.1 考虑深度学习	65
3.7.2 测试时的批量归一化	67
3.7.3 内部协变量偏移	69
第 4 章 卷积神经网络	72
4.1 观察 1: 检测模式不需要整张图像	74
4.2 简化 1: 感受野	74
4.3 观察 2: 同样的模式可能会出现在图像的不同区域	78

4.4	简化 2: 共享参数	78
4.5	简化 1 和 2 的总结	79
4.6	观察 3: 下采样不影响模式检测	83
4.7	简化 3: 汇聚	84
4.8	卷积神经网络的应用: 下围棋	86
第 5 章	循环神经网络	90
5.1	独热编码	90
5.2	什么是 RNN?	91
5.3	RNN 架构	92
5.4	其他 RNN	93
5.4.1	Elman 网络 & Jordan 网络	93
5.4.2	双向循环神经网络	94
5.4.3	长短期记忆网络	95
5.4.4	LSTM 举例	96
5.4.5	LSTM 运算示例	97
5.5	LSTM 原理	99
5.6	RNN 学习方式	101
5.7	如何解决 RNN 梯度消失或者爆炸	103
5.8	RNN 其他应用	104
5.8.1	多对一序列	104
5.8.2	多对多序列	104
5.8.3	序列到序列	106
第 6 章	自注意力机制	109
6.1	输入是向量序列的情况	109
6.1.1	类型 1: 输入与输出数量相同	111
6.1.2	类型 2: 输入是一个序列, 输出是一个标签	112
6.1.3	类型 3: 序列到序列	112
6.2	自注意力的运作原理	113
6.3	多头注意力	121
6.4	位置编码	122
6.5	截断自注意力	124
6.6	自注意力与卷积神经网络对比	125
6.7	自注意力与循环神经网络对比	127
第 7 章	Transformer	130
7.1	序列到序列模型	130
7.1.1	语音识别、机器翻译与语音翻译	130
7.1.2	语音合成	131
7.1.3	聊天机器人	131
7.1.4	问答任务	132

7.1.5	句法分析	132
7.1.6	多标签分类	133
7.2	Transformer 结构	134
7.3	Transformer 编码器	135
7.4	Transformer 解码器	136
7.4.1	自回归解码器	137
7.4.2	非自回归解码器	140
7.5	编码器-解码器注意力	141
7.6	Transformer 的训练过程	141
7.7	序列到序列模型训练常用技巧	142
7.7.1	复制机制	143
7.7.2	引导注意力	143
7.7.3	束搜索	143
7.7.4	加入噪声	145
7.7.5	使用强化学习训练	145
7.7.6	计划采样	145
第 8 章	生成模型	147
8.1	生成对抗网络	147
8.1.1	生成器	147
8.1.2	判别器	150
8.2	生成器与判别器的训练过程	151
8.3	GAN 的应用案例	152
8.4	GAN 的理论介绍	154
8.5	WGAN 算法	157
8.6	训练 GAN 的难点与技巧	161
8.7	GAN 的性能评估方法	163
8.8	条件型生成	167
8.9	Cycle GAN	168
第 9 章	扩散模型	173
第 10 章	自监督学习	177
10.1	来自 Transformers 的双向编码器表示 (BERT)	178
10.1.1	BERT 的使用方式	181
10.1.2	BERT 有用的原因	190
10.1.3	BERT 的变种	194
10.2	生成式预训练 (GPT)	197
第 11 章	自编码器	202
11.1	自编码器的概念	202
11.2	为什么需要自编码器?	203

11.3 去噪自编码器	205
11.4 自编码器应用之特征解耦	205
11.5 自编码器应用之离散隐表征	208
11.6 自编码器的其他应用	210
第 12 章 对抗攻击	212
12.1 对抗攻击简介	212
12.2 如何进行网络攻击	213
12.3 快速梯度符号法	216
12.4 白盒攻击与黑盒攻击	216
12.5 其他模态数据被攻击案例	220
12.6 现实世界中的攻击	221
12.7 防御方式中的被动防御	223
12.8 防御方式中的主动防御	225
第 13 章 迁移学习	227
13.1 领域偏移	227
13.2 领域自适应	228
13.3 领域泛化	233
第 14 章 强化学习	236
14.1 强化学习应用	237
14.1.1 玩电子游戏	237
14.1.2 下围棋	238
14.2 强化学习框架	239
14.2.1 第 1 步: 未知函数	239
14.2.2 第 2 步: 定义损失	240
14.2.3 第 3 步: 优化	241
14.3 评价动作的标准	244
14.3.1 使用即时奖励作为评价标准	244
14.3.2 使用累积奖励作为评价标准	245
14.3.3 使用折扣累积奖励作为评价标准	245
14.3.4 使用折扣累积奖励减去基线作为评价标准	247
14.3.5 Actor-Critic	249
14.3.6 优势 Actor-Critic	254
第 15 章 元学习	256
15.1 元学习的概念	256
15.2 元学习的三个步骤	257
15.3 元学习与机器学习	259
15.4 元学习的实例算法	261
15.5 元学习的应用	264

第 16 章 终身学习	266
16.1 灾难性遗忘	266
16.2 终身学习评估方法	269
16.3 终身学习的主要解法	270
第 17 章 网络压缩	273
17.1 网络剪枝	273
17.2 知识蒸馏	278
17.3 参数量化	281
17.4 网络架构设计	282
17.5 动态计算	286
第 18 章 可解释性人工智能	291
18.1 可解释性人工智能的重要性	291
18.2 决策树模型的可解释性	292
18.3 可解释性机器学习的目标	293
18.4 可解释性机器学习中的局部解释	293
18.5 可解释性机器学习中的全局解释	299
18.6 扩展与小结	302
第 19 章 ChatGPT	304
19.1 ChatGPT 简介和功能	304
19.2 对于 ChatGPT 的误解	304
19.3 ChatGPT 背后的关键技术——预训练	307
19.4 ChatGPT 带来的研究问题	311
第 20 章 ICLR 2025 Oral 单卡 3090 纯视觉玩 Minecraft	314
20.1 简介	314
20.2 主要创新点和贡献	315
20.3 方法	315
20.3.1 功用性图计算	315
20.3.2 快速功用性图生成	316
20.3.3 根据功用性图计算内在奖励以及评估跳跃式状态转换的必要性	316
20.3.4 长短期世界模型	317
20.3.5 在长短期想象序列上进行行为学习	317
20.4 实验结果	318
20.5 总结	320
术语	321

第 1 章 机器学习基础

首先简单介绍一下**机器学习 (Machine Learning, ML)** 和**深度学习 (Deep Learning, DL)** 的基本概念。机器学习, 顾名思义, 机器具备有学习的能力。具体来讲, 机器学习就是让机器具备找一个函数的能力。机器具备找函数的能力以后, 它可以做很多事。比如语音识别, 机器听一段声音, 产生这段声音对应的文字。我们需要的是一个函数, 该函数的输入是声音信号, 输出是这段声音信号的内容。这个函数显然非常复杂, 人类难以把它写出来, 因此想通过机器的力量把这个函数自动找出来。还有好多的任务需要找一个很复杂的函数, 以图像识别为例, 图像识别函数的输入是一张图片, 输出是这个图片里面的内容。AlphaGo 也可以看作是一个函数, 机器下围棋需要的就是一个函数, 该函数的输入是棋盘上黑子跟白子的位置, 输出是机器下一步应该落子的位置。

随着要找的函数不同, 机器学习有不同的类别。假设要找的函数的输出是一个数值, 一个标量 (scalar), 这种机器学习的任务称为回归。举个回归的例子, 假设机器要预测未来某一个时间的 PM2.5 的数值。机器要找一个函数 f , 其输入是可能是种种跟预测 PM2.5 有关的指数, 包括今天的 PM2.5 的数值、平均温度、平均的臭氧浓度等等, 输出是明天中午的 PM2.5 的数值, 找这个函数的任务称为**回归 (regression)**。

除了回归以外, 另一个常见的任务是**分类 (classification)**。分类任务要让机器做选择题。人类先准备好一些选项, 这些选项称为类别 (class), 现在要找的函数的输出就是从设定好的选项里面选择一个当作输出, 该任务称为分类。举个例子, 每个人都有邮箱账户, 邮箱账户里面有一个函数, 该函数可以检测一封邮件是否为垃圾邮件。分类不一定只有两个选项, 也可以有多个选项。

AlphaGo 也是一个分类的问题, 如果让机器下围棋, 做一个 AlphaGo, 给出的选项与棋盘的位置有关。棋盘上有 19×19 个位置, 机器下围棋其实是一个有 19×19 个选项的选择题。机器找一个函数, 该函数的输入是棋盘上黑子跟白子的位置, 输出就是从 19×19 个选项里面, 选出一个正确的选项, 从 19×19 个可以落子的位置里面, 选出下一步应该要落子的位置。

在机器学习领域里面, 除了回归跟分类以外, 还有结构化学习 (structured learning)。机器不只是要做选择题或输出一个数字, 而是产生一个有结构的物体, 比如让机器画一张图, 写一篇文章。这种叫机器产生有结构的东西的问题称为结构化学习。

1.1 案例学习

以视频的点击次数预测为例介绍下机器学习的运作过程。假设有人想要通过视频平台赚钱, 他会在意频道有没有流量, 这样他才会知道他的获利。假设后台可以看到很多相关的信息, 比如: 每天点赞的人数、订阅人数、观看次数。根据一个频道过往所有的信息可以预测明天的观看次数。找一个函数, 该函数的输入是后台的信息, 输出是隔天这个频道会有的总观看的次数。

机器学习找函数的过程, 分成 3 个步骤。第一个步骤是写出一个带有未知参数的函数 f , 其能预测未来观看次数。比如将函数写成

$$y = b + wx_1 \quad (1.1)$$

其中, y 是准备要预测的东西, 要预测的是今天 (2 月 26 日) 这个频道总共观看的人, y 就假设是今天总共的观看次数。 x_1 是这个频道, 前一天 (2 月 25 日) 总共的观看次数, y 跟 x_1 都

是数值, b 跟 w 是未知的参数, 它是准备要通过数据去找出来的, w 跟 b 是未知的, 只是隐约地猜测。猜测往往来自于对这个问题本质上的了解, 即领域知识 (domain knowledge)。机器学习就需要一些领域知识。这是一个猜测, 也许今天的观看次数, 总是会跟昨天的观看次数有点关联, 所以把昨天的观看次数, 乘上一个数值, 但是总是不会一模一样, 所以再加上一个 b 做修正, 当作是对于 2 月 26 日, 观看次数的预测, 这是一个猜测, 它不一定是对的, 等一下回头会再来修正这个猜测。总之, $y = b + w * x_1$, 而 b 跟 w 是未知的。带有未知的参数 (parameter) 的函数称为模型 (model)。模型在机器学习里面, 就是一个带有未知的参数的函数, 特征 (feature) x_1 是这个函数里面已知的, 它是来自于后台的信息, 2 月 25 日点击的总次数是已知的, 而 w 跟 b 是未知的参数。 w 称为权重 (weight), b 称为偏置 (bias)。这个是第一个步骤。

第 2 个步骤是定义损失 (loss), 损失也是一个函数。这个函数的输入是模型里面的参数, 模型是 $y = b + w * x_1$, 而 b 跟 w 是未知的, 损失是函数 $L(b, w)$, 其输入是模型参数 b 跟 w 。损失函数输出的值代表, 现在如果把这一组未知的参数, 设定某一个数值的时候, 这笔数值好还是不好。举一个具体的例子, 假设未知的参数的设定是 $b = 500$, $w = 1$, 预测未来的观看次数的函数就变成 $y = 500 + x_1$ 。要从训练数据来进行计算损失, 在这个问题里面, 训练数据是这一个频道过去的观看次数。举个例子, 从 2017 年 1 月 1 日到 2020 年 12 月 31 日的观看次数 (此处的数字是随意生成的) 如图 1.1 所示, 接下来就可以计算损失。

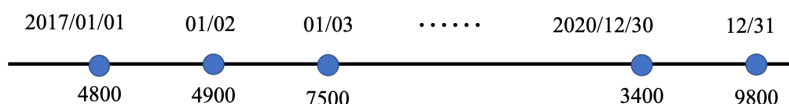


图 1.1 2017 年 1 月 1 日到 2020 年 12 月 31 日的观看次数

把 2017 年 1 月 1 日的观看次数, 代入这一个函数里面

$$\hat{y} = 500 + 1x_1 \quad (1.2)$$

可以判断 $b = 500$, $w = 1$ 的时候, 这个函数有多棒。 x_1 代入 4800, 预测隔天实际上的观看次数结果为 $\hat{y} = 5300$, 真正的结果是 4900, 真实的值称为标签 (label), 它高估了这个频道可能的点击次数, 可以计算一下估测的值 $\hat{y}$ 跟真实值 y 的差距 e 。计算差距其实不只一种方式, 比如取绝对值:

$$e_1 = |y - \hat{y}| = 400 \quad (1.3)$$

我们不是只能用 1 月 1 日, 来预测 1 月 2 日的值, 可以用 1 月 2 日的值, 来预测 1 月 3 日的值。根据 1 月 2 日的观看次数, 预测的 1 月 3 日的观看次数的, 值是 5400。接下来计算 5400 跟跟标签 (7500) 之间的差距, 低估了这个频道。在 1 月 3 日的时候的观看次数, 才可以算出:

$$e_2 = |y - \hat{y}| = 2100 \quad (1.4)$$

我们可以算过这 3 年来, 每一天的预测的误差, 这 3 年来每一天的误差, 通通都可以算出来, 每一天的误差都可以得到 e 。接下来把每一天的误差, 通通加起来取得平均, 得到损失 L

$$L = \frac{1}{N} \sum_n e_n \quad (1.5)$$

其中, N 代表训练数据的个数, 即 3 年来的训练数据, 就 365 乘以 3, 计算出一个 L , L 是每一笔训练数据的误差 e 相加以后的结果。 L 越大, 代表现在这一组参数越不好, L 越小, 代表现在这一组参数越好。

估测的值跟实际的值之间的差距, 其实有不同的计算方法, 计算 y 与 $\hat{y}$ 之间绝对值的差距, 如式 (1.6) 所示, 称为**平均绝对误差 (Mean Absolute Error, MAE)**。

$$e = |\hat{y} - y| \quad (1.6)$$

如果算 y 与 $\hat{y}$ 之间平方的差距, 如式 (1.7) 所示, 则称为**均方误差 (Mean Squared Error, MSE)**。

$$e = (\hat{y} - y)^2 \quad (1.7)$$

有一些任务中 y 和 $\hat{y}$ 都是概率分布, 这个时候可能会选择**交叉熵 (cross entropy)**, 这个是机器学习的第 2 步。刚才举的那些数字不是真正的例子, 以下的数字是真实的例子, 是这个频道真实的后台的数据, 所计算出来的结果。可以调整不同的 w 和不同的 b , 求取各种 w 和各种 b , 组合起来以后, 我们可以为不同的 w 跟 b 的组合, 都去计算它的损失, 就可以画出图 1.2 所示的等高线图。在这个等高线图上面, 越偏红色系, 代表计算出来的损失越大, 就代表这一组 w 跟 b 越差。如果越偏蓝色系, 就代表损失越小, 就代表这一组 w 跟 b 越好, 拿这一组 w 跟 b , 放到函数里面, 预测会越精准。假设 $w = -0.25, b = -500$, 这代表这个频道每天看的人越来越少, 而且损失这么大, 跟真实的情况不太合。如果 $w = 0.75, b = 500$, 估测会比较精准。如果 w 代一个很接近 1 的值, b 带一个小小的值, 比如说 100 多, 这个时候估测是最精准的, 这跟大家的预期可能是比较接近的, 就是拿前一天的点击的总次数, 去预测隔天的点击的总次数, 可能前一天跟隔天的点击的总次数是差不多的, 因此 w 设 1, b 设一个小一点的数值, 也许估测就会蛮精准的。如图 1.2 所示的等高线图, 就是试了不同的参数, 计算它的损失, 画出来的等高线图称为**误差表面 (error surface)**。这是机器学习的第 2 步。

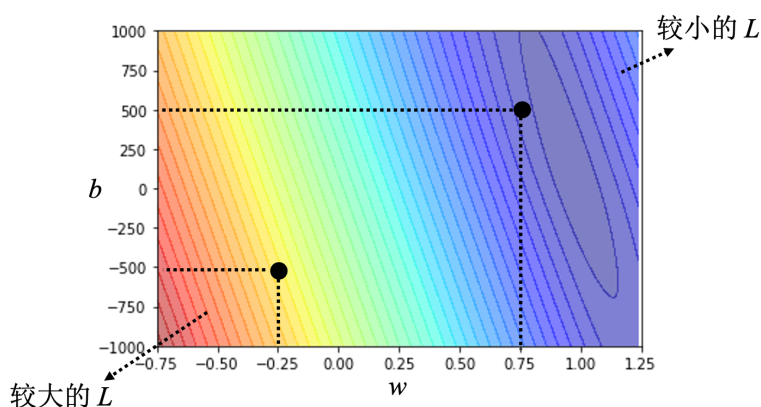


图 1.2 误差表面

接下来进入机器学习的第 3 步: 解一个最优化问题。找一个 w 跟 b , 把未知的参数找一个数值出来, 看代哪一个数值进去可以让损失 L 的值最小, 就是要找的 w 跟 b , 这个可以让损失最小的 w 跟 b 称为 w^* 跟 b^* 代表它们是最好的一组 w 跟 b , 可以让损失的值最小。**梯度下降 (gradient descent)** 是经常会使用优化的方法。为了要简化起见, 先假设只有一个未知的参数 w , b 是已知的。 w 代不同的数值的时候, 就会得到不同的损失, 这一条曲线就

是误差表面，只是刚才在前一个例子里面，误差表面是 2 维的，这边只有一个参数，所以这个误差表面是 1 维的。怎么样找一个 w 让损失的值最小呢？如图 1.3 所示，首先要随机选取一个初始的点 w^0 。接下来计算 $\frac{\partial L}{\partial w}|_{w=w^0}$ ，在 w 等于 w^0 的时候，损失关于参数 w 的偏导数。计算在这一个点，在 w^0 这个位置的误差表面的切线斜率，也就是这一条蓝色的虚线，它的斜率，如果这一条虚线的斜率是负的，代表说左边比较高，右边比较低。在这个位置附近，左边比较高，右边比较低。如果左边比较高右边比较低的话，就把 w 的值变大，就可以让损失变小。如果算出来的斜率是正的，就代表左边比较低右边比较高。左边比较低右边比较高，如果左边比较低右边比较高的话，就代表把 w 变小了， w 往左边移，可以让损失的值变小。这个时候就应该把 w 的值变小。我们可以想像说有一个人站在这个地方，他左右环视一下，算微分就是左右环视，它会知道左边比较高还是右边比较高，看哪边比较低，它就往比较低的地方跨出一步。这一步的步伐的大小取决于两件事情：

- 第一件事情是这个地方的斜率，斜率大步伐就跨大一点，斜率小步伐就跨小一点。
- 另外，**学习率 (learning rate) η** 也会影响步伐大小。学习率是自己设定的，如果 η 设大一点，每次参数更新就会量大，学习可能就比较快。如果 η 设小一点，参数更新就很慢，每次只会改变一点点参数的数值。这种在做机器学习，需要自己设定，不是机器自己找出来的，称为**超参数 (hyperparameter)**。

Q: 为什么损失可以是负的？

A: 损失函数是自己定义的，在刚才定义里面，损失就是估测的值跟正确的值的绝对值。如果根据刚才损失的定义，它不可能是负的。但是损失函数是自己决定的，比如设置一个损失函数为绝对值再减 100，其可能就有负的。这个曲线并不是一个真实的损失，并不是一个真实任务的误差表面。因此这个损失的曲线可以是任何形状。

把 w^0 往右移一步，新的位置为 w^1 ，这一步的步伐是 η 乘上微分的结果，即：

$$w^1 \leftarrow w^0 - \eta \frac{\partial L}{\partial w} \Big|_{w=w^0} \quad (1.8)$$

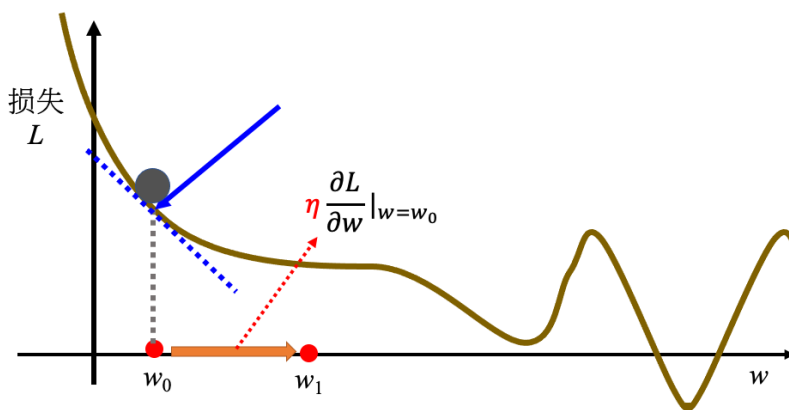


图 1.3 优化过程

接下来反复进行刚才的操作，计算一下 w^1 微分的结果，再决定现在要把 w^1 移动多少，再移动到 w^2 ，再继续反复做同样的操作，不断地移动 w 的位置，最后会停下来。往往有两种情况会停下来。

- 第一种情况是一开始会设定说，在调整参数的时候，在计算微分的时候，最多计算几次。上限可能会设为 100 万次，参数更新 100 万次后，就不再更新了，更新次数也是一个超参数。
- 还有另外一种理想上的，停下来的可能是，当不断调整参数，调整到一个地方，它的微分的值就是这一项，算出来正好是 0 的时候，如果这一项正好算出来是 0，0 乘上学习率 η 还是 0，所以参数就不会再移动位置。假设是这个理想的情况，把 w^0 更新到 w^1 ，再更新到 w^2 ，最后更新到 w^T 有点卡， w^T 卡住了，也就是算出来这个微分的值是 0 了，参数的位置就不会再更新。

梯度下降有一个很大的问题，没有找到真正最好的解，没有找到可以让损失最小的 w 。在图 1.4 所示的例子里面，把 w 设定在最右侧红点附近这个地方可以让损失最小。但如果在梯度下降中， w^0 是随机初始的位置，也很可能走到 w^T 这里，训练就停住了，无法再移动 w 的位置。右侧红点这个位置是真的可以让损失最小的地方，称为**全局最小值 (global minima)**，而 w^T 这个地方称为**局部最小值 (local minima)**，其左右两边都比这个地方的损失还要高一点，但是它不是整个误差表面上面的最低点。

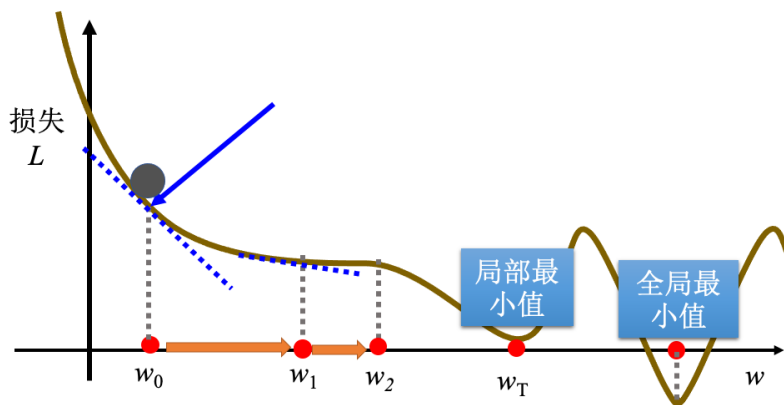


图 1.4 局部最小值

所以常常可能会听到有人讲到梯度下降不是个好方法，这个方法会有局部最小值的问题，无法真的找到全局最小值。事实上局部最小值是一个假问题，在做梯度下降的时候，真正面对的难题不是局部最小值。有两个参数的情况下使用梯度下降，其实跟刚才一个参数没有什么不同。如果一个参数没有问题的话，可以很快的推广到两个参数。

假设有两个参数，随机初始值为 w^0, b^0 。要计算损失关于 w, b 的偏导数，计算在 $w = w^0$ 的位置， $b = b^0$ 的位置，要计算 L 关于 w 的偏导数，计算 L 关于 b 的偏导数：

$$\left. \frac{\partial L}{\partial b} \right|_{w=w^0, b=b^0} \quad (1.9)$$

$$\left. \frac{\partial L}{\partial w} \right|_{w=w^0, b=b^0}$$

计算完后更新 w 跟 b ，把 w^0 减掉学习率乘上微分的结果得到 w^1 ，把 b^0 减掉学习率乘

上微分的结果得到 b^1 。

$$\begin{aligned} w^1 &\leftarrow w^0 - \eta \left. \frac{\partial L}{\partial w} \right|_{w=w^0, b=b^0} \\ b^1 &\leftarrow b^0 - \eta \left. \frac{\partial L}{\partial b} \right|_{w=w^0, b=b^0} \end{aligned} \quad (1.10)$$

在深度学习框架里面，比如 PyTorch 里面，算微分都是程序自动帮计算的。就是反复同样的步骤，就不断的更新 w 跟 b ，期待最后，可以找到一个最好的 w ， w^* 跟最好的 b^* 。如图 1.5 所示，随便选一个初始的值，先计算一下 L 关于 w 的偏导数，跟计算一下 L 关于 b 的偏导数，接下来更新 w 跟 b ，更新的方向就是 $\partial L / \partial w$ ，乘以 η 再乘以一个负号， $\partial L / \partial b$ ，算出这个微分的值，就可以决定更新的方向，可以决定 w 要怎么更新。把 w 跟 b 更新的方向结合起来，就是一个向量，就是红色的箭头，再计算一次微分，再决定要走什么样的方向，把这个微分的值乘上学习率，再乘上负号，我们就知道红色的箭头要指向那里，就知道如何移动 w 跟 b 的位置，一直移动，期待最后可以找出一组不错的 w, b 。实际上真的用梯度下降，进行一番计算以后，这个是真的数据，算出来的最好的 $w^* = 0.97, b^* = 100$ ，跟猜测蛮接近的。因为 x_1 的值可能跟 y 很接近，所以这个 w 就设一个接近 1 的值， b 就设一个比较偏小的值。损失 $L(w^*, b^*)$ 算一下是 480，也就是在 2017 到 2020 年的数据上，如果使用这一个函数， b 代 100， w 代 0.97，平均的误差是 480，其预测的观看次数误差，大概是 500 人左右。

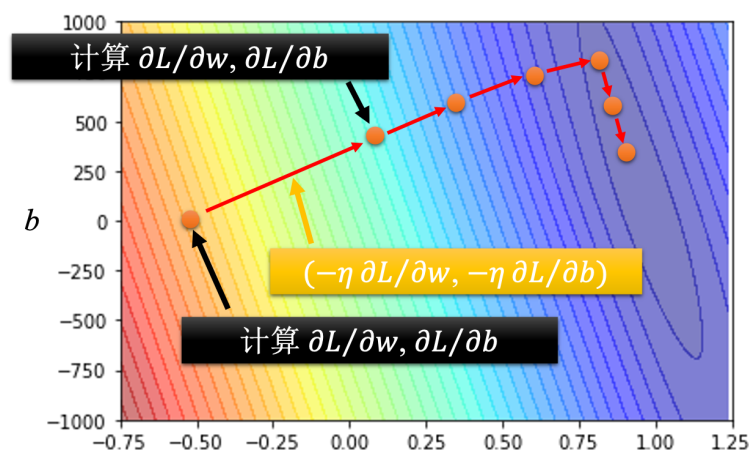


图 1.5 梯度下降优化的过程

1.2 线性模型

w 跟 b 的值刚才已经找出来的，这组 w 跟 b 可以让损失小到 480。在已经知道答案的数据上去计算损失，2017 到 2020 年每天的观看次数是已知的。所以假装不知道隔天的观看次数，拿这一个函数来进行预测，发现误差是 480。接下来使用这个函数预测未来的观看次数。预测从 2021 年开始每一天都拿这个函数去预测次日的观看人次：用 2020 年的 12 月 31 日的观看人次预测 2021 年 1 月 1 日的观看人次，用 2021 年 1 月 1 日的观看人次预测 1 月 2 日的观看人次，用 1 月 2 日的观看人次去预测 1 月 3 日的观看人次……每天都做这件事，一直做到 2 月 14 日，得到平均的值，在 2021 年数据上的误差值用 L' 来表示，它是 0.58，所以在有看过的数据上，在训练数据上，误差值是比较小的，在 2021 年的数据上，看起来误差值

是比较大的，每一天的平均误差有 580 人左右，600 人左右。如图 1.6 所示，横轴是代表的是时间，所以 0 这个点代表的是 2021 年 1 月 1 日，最右边点代表的是 2021 年 2 月 14 日，纵轴就是观看的人次，这边是用千人当作单位。红色线是真实的观看人次，蓝色线是机器用这一个函数预测出来的观看人次。蓝色的线几乎就是红色的线往右平移一天而已，这很合理，因为 x_1 也就是前一天的观看人次，跟隔天观看人次的，要怎么拿前一天的观看人次，去预测隔天的观看人次呢，前一天观看人次乘以 0.97，加上 100 加上 100，就是隔天的观看人次。机器几乎就是拿前一天的观看人次来预测隔天的观看人次。这个真实的数据有一个很神奇的现象，它是有周期性的，它每隔 7 天就会有两天特别低（周五和周六），两天观看的人特别少，每隔 7 天，就是一个循环。目前的模型不太行，它只能看前一天。每隔 7 天它一个循环，如果一个模型参考前 7 天的数据，把 7 天前的数据，直接复制到拿来当作预测的结果，也许预测的会更准也说不定，所以我们就要修改一下模型。通常一个模型的修改，往往来自于对这个问题的理解，即领域知识。

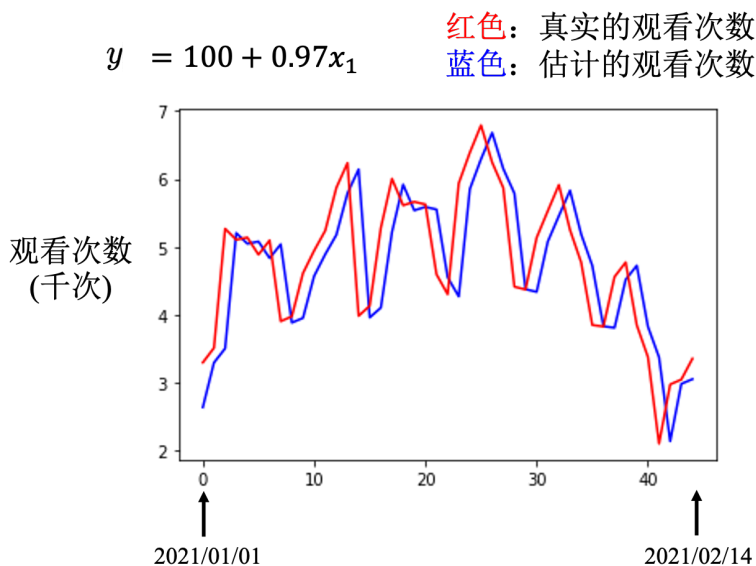


图 1.6 预估曲线图

一开始，对问题完全不理解的时候，胡乱写一个

$$y = b + wx_1 \quad (1.11)$$

并没有做得特别好。接下来我们观察了真实的数据以后，得到一个结论是，每隔 7 天有一个循环。所以要把前 7 天的观看人次都列入考虑，写了一个新的模型

$$y = b + \sum_{j=1}^7 w_j x_j \quad (1.12)$$

其中 x_j 代表第 j 天的观看测试，也就是 7 天前的数据，通通乘上不同的权重 w_j ，加起来，再加上偏置得到预测的结果。使用该模型预测，其在训练数据上的损失是 380，而只考虑 1 天的模型在训练数据上的损失是 480。因为其考虑了 7 天，所以在训练数据上会得到比较低的损失。考虑了比较多的信息，在训练数据上应该要得到更好的、更低的损失。在没有看到的数据上的损失有比较好是 490。只考虑 1 天的误差是 580，考虑 7 天的误差是 490。用梯度下降，算出 w 跟 b 的最优值如表 1.1 所示。

表 1.1 w 和 b 的最优值

b	w_1^*	w_2^*	w_3^*	w_4^*	w_5^*	w_6^*	w_7^*
50	0.79	-0.31	0.12	-0.01	-0.10	0.30	0.18

机器的逻辑是前一天跟要预测的隔天的数值的关系很大, 所以 w_1^* 是 0.79, 不过它知道, 如果是前两天前四天前五天, 它的值会跟未来要预测的, 隔天的值是成反比的, 所以 w_2, w_4, w_5 最佳的值 (让训练数据上的损失为 380 的值) 是负的。但是 w_1, w_3, w_6, w_7 是正的, 考虑前 7 天的值, 其实可以考虑更多天, 本来是考虑前 7 天, 可以考虑 28 天, 即

$$y = b + \sum_{j=1}^{28} w_j x_j. \quad (1.13)$$

28 天是一个月, 考虑前一个月每一天的观看人次, 去预测隔天的观看人次, 训练数据上是 330。在 2021 年的数据上, 损失是 460, 看起来又更好一点。如果考虑 56 天, 即

$$y = b + \sum_{j=1}^{56} w_j x_j \quad (1.14)$$

在训练数据上损失是 320, 在没看过的数据上损失还是 460。考虑更多天没有办法再更降低损失了。看来考虑天数这件事, 也许已经到了一个极限。这些模型都是把输入的特征 x 乘上一个权重, 再加上一个偏置就得到预测的结果, 这样的模型称为**线性模型 (linear model)**。接下来会看如何把线性模型做得更好。

1.2.1 分段线性曲线

线性模型也许过于简单, x_1 跟 y 可能中间有比较复杂的关系, 如图 1.7 所示。对于线性模型, x_1 跟 y 的关系就是一条直线, 随着 x_1 越来越高, y 就应该越来越大。设定不同的 w 可以改变这条线的斜率, 设定不同的 b 可以改变这一条蓝色的直线跟 y 轴的交叉点。但是无论如何改 w 跟 b , 它永远都是一条直线, 永远都是 x_1 越大, y 就越大, 前一天观看的次数越多, 隔天的观看次数就越多。但现实中也许在 x_1 小于某一个数值的时候, 前一天的观看次数跟隔天的观看次数是成正比; 也许当 x_1 大于一个数值的时候, x_1 太大, 前天观看的次数太高, 隔天观看次数就会变少; 也许 x_1 跟 y 中间, 有一个比较复杂的、像红色线一样的关系。但不管如何设置 w 跟 b , 永远制造不出红色线, 永远无法用线性模型制造红色线。显然线性模型有很大的限制, 这一种来自于模型的限制称为模型的偏差, 无法模拟真实的情况。

所以需要写一个更复杂的、更有灵活性的、有未知参数的函数。红色的曲线可以看作是一个常数再加上一群 Hard Sigmoid 函数。Hard Sigmoid 函数的特性是当输入的值, 当 x 轴的值小于某一个阈值 (某个定值) 的时候, 大于另外一个定值阈值的时候, 中间有一个斜坡。所以它是先水平的, 再斜坡, 再水平的。所以红色的线可以看作是一个常数项加一大堆的蓝色函数 (Hard Sigmoid)。常数项设成红色的线跟 x 轴的交点一样大。常数项怎么加上蓝色函数后, 变成红色的这一条线? 蓝线 1 函数斜坡的起点, 设在红色函数的起始的地方, 第 2 个斜坡的终点设在第一个转角处, 让第 1 个蓝色函数的斜坡和红色函数的斜坡的斜率是一样的, 这个时候把 0+1 就可以得到红色曲线左侧的线段。接下来, 再加第 2 个蓝色的函数, 所以第 2 个蓝色函数的斜坡就在红色函数的第一个转折点到第 2 个转折点之间, 让第 2 个蓝色函数

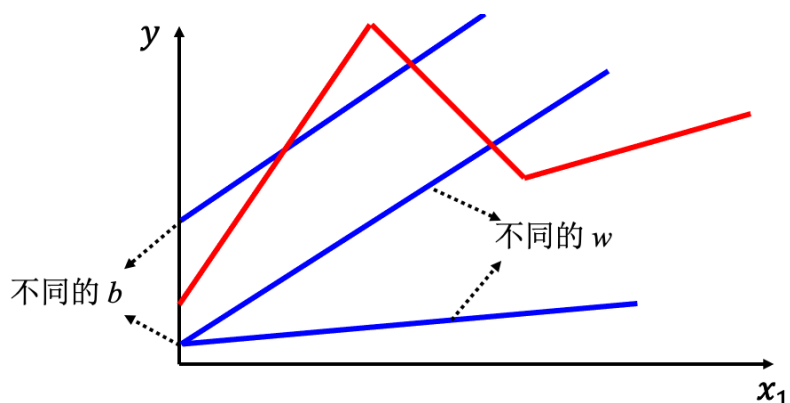


图 1.7 线性模型的局限性

的斜率跟红色函数的斜率一样，这个时候把 $0+1+2$ ，就可以得到红色函数左侧和中间的线段。接下来第 3 个部分，第 2 个转折点之后的部分，就加第 3 个蓝色的函数，第 3 个蓝色的函数坡度的起始点设的跟红色函数转折点一样，蓝色函数的斜率设的跟红色函数斜率一样，接下来把 $0+1+2+3$ 全部加起来，就得到完整红色的线。

所以红色线，即分段线性曲线 (piecewise linear curve) 可以看作是一个常数，再加上一堆蓝色的函数。分段线性曲线可以用常数项加一大堆的蓝色函数组合出来，只是用的蓝色函数不一定一样。要有很多不同的蓝色函数，加上一个常数以后就可以组出这些分段线性曲线。如果分段线性曲线越复杂，转折的点越多，所需的蓝色函数就越多。

红色曲线 = 常数 + 一组  的和

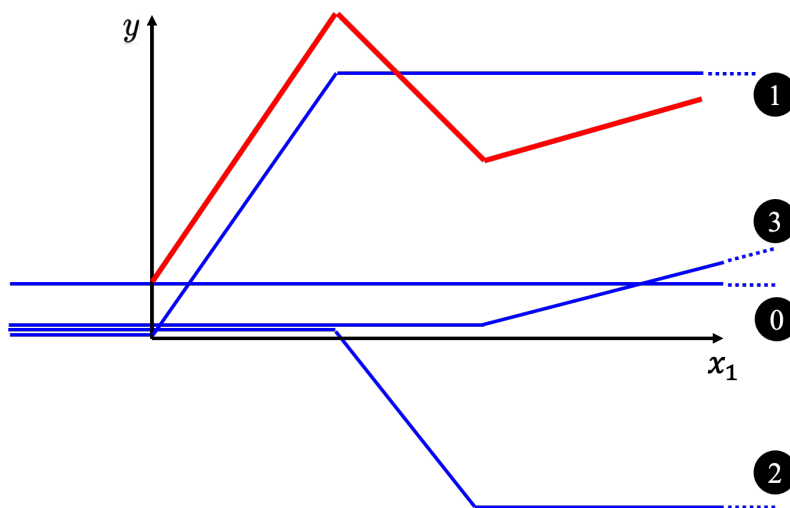


图 1.8 构建红色曲线

也许要考虑的 x 跟 y 的关系不是分段线性曲线，而是如图 1.9 所示的曲线。可以在这样的曲线上，先取一些点，再把这些点点起来，变成一个分段线性曲线。而这个分段线性曲线跟原来的曲线，它会非常接近，如果点取的够多或点取的位置适当，分段线性曲线就可以逼近这一个连续的曲线，就可以逼近有角度的、有弧度的这一条曲线。所以可以用分段线性曲线去逼近任何的连续的曲线，而每个分段线性曲线都可以用一大堆蓝色的函数组合起来。也就

是说，只要有足够的蓝色函数把它加起来，就可以变成任何连续的曲线。

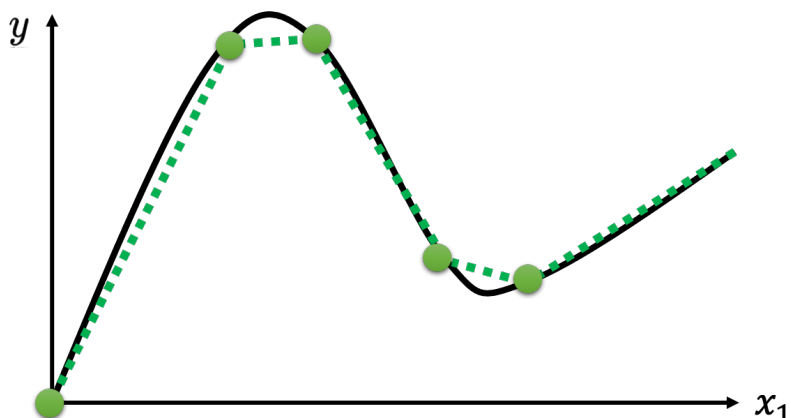


图 1.9 分段曲线可以逼近任何连续曲线

假设 x 跟 y 的关系非常复杂也没关系，就想办法写一个带有未知数的函数。直接写 Hard Sigmoid 不是很容易，但是可以用一条曲线来理解它，用 Sigmoid 函数来逼近 Hard Sigmoid，如图 1.10 所示。Sigmoid 函数的表达式为

$$y = c \frac{1}{1 + e^{-(b+wx_1)}}$$

其横轴输入是 x_1 ，输出是 y ， c 为常数。

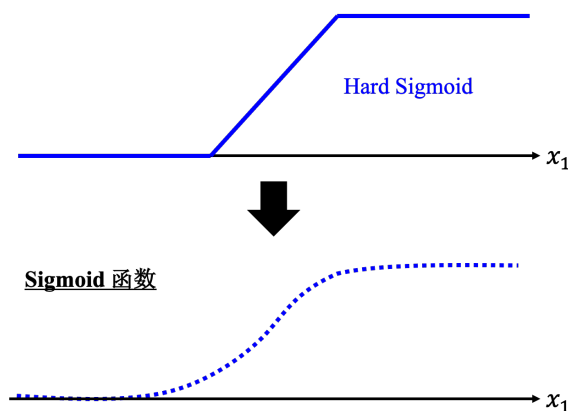


图 1.10 使用 Sigmoid 逼近 Hard Sigmoid

如果 x_1 的值，趋近于无穷大的时候， $e^{-(b+wx_1)}$ 这一项就会消失，当 x_1 非常大的时候，这一条就会收敛在高度为 c 的地方。如果 x_1 负的非常大的时候，分母的地方就会非常大， y 的值就会趋近于 0。

所以可以用这样子的一个函数逼近这一个蓝色的函数，即 Sigmoid 函数，Sigmoid 函数就是 S 型的函数。因为它长得是有点像是 S 型，所以叫它 Sigmoid 函数。

为了简洁，去掉了指数的部分，蓝色函数的表达式为

$$y = c\sigma(b + wx_1) \quad (1.15)$$

所以可以用 Sigmoid 函数逼近 Hard Sigmoid 函数。

$$y = c \frac{1}{1 + e^{-(b+wx_1)}} \quad (1.16)$$

调整这里的 b 、 w 和 c 可以制造各种不同形状的 Sigmoid 函数, 用各种不同形状的 Sigmoid 函数去逼近 Hard Sigmoid 函数。如图 1.11 所示, 如果改 w , 就会改变斜率, 就会改变斜坡的坡度。如果改了 b , 就可以把这这个 Sigmoid 函数左右移动; 如果改 c , 就可以改变它的高度。所以只要有不同的 w 不同的 b 不同的 c , 就可以制造出不同的 Sigmoid 函数, 把不同的 Sigmoid 函数叠起来以后就可以去逼近各种不同的分段线性函数; 分段线性函数可以拿来近似各种不同的连续的函数。

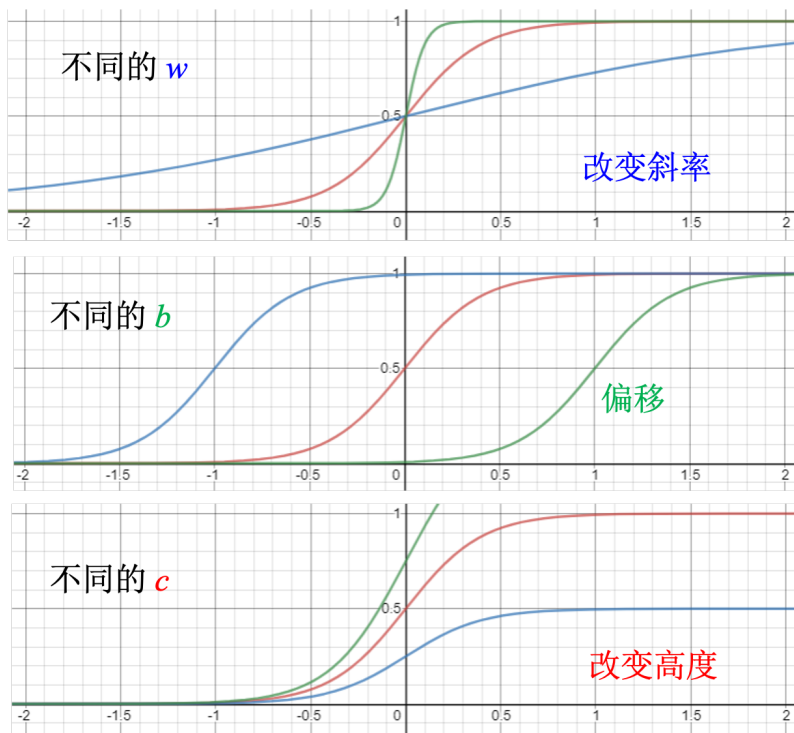


图 1.11 调整参数, 制造不同的 Sigmoid 函数

如图 1.12 所示, 红色这条线就是 0 加 $1+2+3$, 而 1、2、3 都是蓝色的函数, 其都可写成 $(b + wx_1)$, 去做 Sigmoid 再乘上 c_i , 只是 1、2、3 的 w 、 b 、 c 不同。

$$y = b + \sum_i c_i \sigma(b_i + w_i x_1) \quad (1.17)$$

所以这边每一个式子都代表了一个不同蓝色的函数, 求和就是把不同的蓝色的函数相加, 再加一个常数 b 。假设里面的 b 跟 w 跟 c , 它是未知的, 它是未知的参数, 就可以设定不同的 b 跟 w 跟 c , 就可以制造不同的蓝色的函数, 制造不同的蓝色的函数叠起来以后, 就可以制造出不同的红色的曲线, 就可以制造出不同的分段线性曲线, 逼近各式各样不同的连续函数。

此外, 我们可以不只用一个特征 x_1 , 可以用多个特征代入不同的 c, b, w , 组合出各种不同的函数, 从而得到更有灵活性 (**flexibility**) 的函数, 如图 1.13 所示。用 j 来代表特征的编号。如果要考虑前 28 天, j 就是 1 到 28。

直观来讲, 先考虑一下 j 就是 1、2、3 的情况, 就是只考虑 3 个特征。举个例子, 只考虑前一天、前两天跟前 3 天的情况, 所以 j 等于 1,2,3, 所以输入就是 x_1 代表前一天的观看